МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»

Навчально-науковий фізико-технічний інститут

Кафедра інформаційної безпеки

ЗВІТ

з дисципліни «Проєктування високонавантажених систем»

Лабораторна робота № 4. Реплікація MongoDB: відмовостійкість і лічильник

Виконав: Вовкотруб Олександр Віталійович
Група: ФБ-61мн

Перевірив: Родіонов А. М.

Київ 2026

Зміст

1 Мета роботи **2**

2 Постановка задачі **2**

3 Хід виконання **2**

3.1 Середовище 2

3.2 Розгортання replica set 3

3.3 Частина I: write concern, read concern і перевибори 4

3.3.1 Сценарій 1: `w: 3` без тайм-ауту 4

3.3.2 Сценарій 2: `w: 3` з `wtimeout: 5000` 5

3.3.3 Сценарій 3: перевибори primary 7

3.4 Клієнт MongoDB і retryable writes 9

3.5 Частина II: програма `mongo-bench` 10

3.6 Частина III: веб-лічильник на MongoDB 10

3.7 Методика вимірювань 10

3.8 Аналітичний ноутбук 11

4 Результати **11**

4.1 Частина I: що показали сценарії 11

4.2 Частина II: час і точність лічильника 12

4.3 Відмова primary: що стається з даними 14

4.4 Частина III: порівняння сховищ веб-лічильника 15

5 Висновки **16**

6 Джерела **17**

1 Мета роботи

Розгорнути replica set MongoDB з трьох вузлів (primary і два secondary) і перевірити на ньому, як write concern і read concern визначають, коли запис вважається виконаним і що з ним стається при відмові вузла. Показати перевибори primary. Виміряти швидкість і точність конкурентного лічильника на `findOneAndUpdate` з `$inc` при `w: 1` і `w: "majority"`, зокрема з раптовою відмовою primary посеред навантаження. Використати MongoDB як сховище веб-лічильника з лабораторної роботи № 1 і порівняти його з пам’яттю, диском, PostgreSQL і Hazelcast.

2 Постановка задачі

Робота складається з трьох частин.

Частина I. Налаштувати реплікацію в конфігурації P-S-S (вузли можуть бути окремими процесами) [1…3] і перевірити:

1. запис з `writeConcern: { w: 3 }` без тайм-ауту, коли один вузол вимкнено; поки запис чекає, увімкнути вузол;
2. те саме зі скінченним `wtimeout`: дочекатися тайм-ауту, перевірити, чи дані записались і чи читаються з `readConcern: "majority"`;
3. перевибори primary після вимкнення поточного primary [4] і те, що після повернення старий primary отримує дані, записані без нього.

Частина II. Колекція з лічильником лайків. 10 окремих клієнтів одночасно збільшують його по 10 000 разів через `findOneAndUpdate` з `$inc` [5], тобто очікується 100 000. Чотири прогони: `w: 1`; `w: "majority"`; `w: 1` з вимкненням primary під час роботи; `w: "majority"` з вимкненням primary. Для кожного виміряти час і перевірити фінальне значення. Рядок підключення має перелічувати всі вузли: `mongodb://h1,h2,h3/?replicaSet=...`.

Частина III. Використати кластер MongoDB як сховище веб-лічильника з роботи № 1 (`findOneAndUpdate`, `w: "majority"`), навантажити його 10 клієнтами по 10 000 запитів і звести в таблицю з пам’яттю, диском, PostgreSQL і Hazelcast (загальний час, пропускна здатність).

Протокол: команди та їхні результати у вигляді знімків екрана, і аналіз.

3 Хід виконання

3.1 Середовище

Три процеси `mongod`, клієнти `mongo-bench`, `counter-server` і `loadgen` працюють на одній машині й спілкуються через loopback. Дані всіх трьох вузлів лежать на одному SSD, тому кожен вузол платить за `fsync` той самий диск. Контейнери не використовувались.

Таблиця 3.1. Апаратне і програмне середовище

Параметр	Значення
CPU	Intel Coffee Lake (інженерний зразок, 2,60 ГГц), 8 ядер, 16 потоків, 0,8…4,4 ГГц, L3 16 МіБ
RAM	62 ГіБ
Диск	NVMe SSD Samsung 970 EVO Plus 1 TB, ext4, розділ заповнений приблизно на 98 %
ОС	Fedora Linux 44, ядро 7.2.11-200.fc44.x86_64
Мова	Swift 6.4, збірка `-c release`
СКБД	MongoDB Community 8.2.12 і mongosh 2.10.0 з Nix flake
Драйвер	MongoKitten 7.16.3, retryable writes реалізовано в роботі
Replica set	`rs0`, 3 вузли `127.0.0.1:21430...21432`, однаковий `priority: 1`, `electionTimeoutMillis: 10000`
WiredTiger	кеш 0,25 ГБ на вузол, журнал увімкнено (за замовчуванням)

3.2 Розгортання replica set

Скрипт `scripts/mongo.sh` керує трьома вузлами. Кожен вузол запускається окремим процесом зі своїм каталогом даних і логом:

```
mongod --replSet rs0 --port 21430 --bind_ip 127.0.0.1 \
    --dbpath .mongodata/node1 --logpath reports/logs/mongo-node1.log \
    --wiredTigerCacheSizeGB 0.25 --quiet --timeStampFormat iso8601-utc \
    --fork
```

При першому запуску скрипт ініціалізує набір з трьох членів [3]:

```
rs.initiate({ _id: "rs0", members: [
    { _id: 0, host: "127.0.0.1:21430" },
    { _id: 1, host: "127.0.0.1:21431" },
    { _id: 2, host: "127.0.0.1:21432" } ] })
```

Клієнти підключаються за рядком, що перелічує всі вузли:

```
mongodb://127.0.0.1:21430,127.0.0.1:21431,127.0.0.1:21432/?replicaSet=rs0
```

Решта команд скрипта: `stop-node N` (SIGTERM, штатна зупинка), `kill-node N` (SIGKILL, раптова відмова), `start-node N`, `status` (стан вузлів з погляду primary), `wait-primary`, `wait-down N` (primary бачить вузол N недоступним), `wait-secondary N` (вузол N став SECONDARY і наздогнав primary) і `shell [N]` (mongosh до набору або напряму до вузла).

3.3 Частина I: write concern, read concern і перевибори

Скрипт `scripts/task4-part1.sh` виконує три сценарії і записує протокол кожного у `reports/logs/task4-part1-{1,2,3}.txt`. Команди оболонки мають мітку часу UTC. Сесії mongosh отримують команди зі stdin, а скрипт ставить кожну введену команду після підказки, яку mongosh для неї надрукував, тому протокол виглядає як інтерактивна сесія. Рисунки 3.1…3.8 є знімками цих протоколів у вигляді вікна терміналу, кожен поділено на дві або три частини.

Після SIGTERM процес mongod 8 виходить лише через 15,0 с. У лозі вузла в цей момент є `Entering quiesce mode for shutdown з quiesceTimeMillis: 15000`. Роль primary передається одразу (сценарій 3).

3.3.1 Сценарій 1: `w: 3` без тайм-ауту

Вузол 3 зупинено, primary позначив його недоступним. У терміналі 2 вставка з `{ w: 3, wtimeout: 0 }` чекає, доки документ запишуть усі три вузли (рис. 3.1). Тим часом у терміналі 1 документ уже читається на primary, зокрема з `readConcern("majority")`, бо його мають 2 вузли з 3. Після `start-node 3` вставка повертає `acknowledged: true` (рис. 3.2).

```
task4-part1-1.txt (1/2)

# Сценарій 1. Один secondary вимкнено, запис з writeConcern { w: 3, wtimeout: 0 } (без тайм-ауту).
# Час у [квадратних дужках] і в ISODate(...) указано в UTC.
# mongosh читає команди зі stdin: текст після підказки rs0 [...]> і є введеною командою.

[17:45:51.038] $ scripts/mongo.sh status
rs0, term 1, seen from node1
node1  127.0.0.1:21430  PRIMARY                 health 1  optime 1793641547:16  (pid 553720)
node2  127.0.0.1:21431  SECONDARY               health 1  optime 1793641547:16  lag 0 s  sync node1  (pid 553814)
node3  127.0.0.1:21432  SECONDARY               health 1  optime 1793641547:16  lag 0 s  sync node1  (pid 553903)

[17:45:53.010] $ scripts/mongo.sh stop-node 3
node 3 stopped (SIGTERM, pid 553903), exited after 15.0 s
[17:46:08.196] $ scripts/mongo.sh wait-down 3
node1 sees node3 as (not reachable/healthy) after 0.5 s (Error connecting to 127.0.0.1:21432 :: caused by :: onInvoke
:: caused by :: Connection refused)
[17:46:09.497] $ scripts/mongo.sh status
rs0, term 1, seen from node1
node1  127.0.0.1:21430  PRIMARY                 health 1  optime 1793641567:1  (pid 553720)
node2  127.0.0.1:21431  SECONDARY               health 1  optime 1793641567:1  lag 0 s  sync node1  (pid 553814)
node3  127.0.0.1:21432  (not reachable/healthy)  health 0  optime 0:0  (process stopped)

# [термінал 2] вставка з w: 3 чекає, доки документ запишуть усі три вузли
[17:46:10.531] $ scripts/mongo.sh shell --quiet --norc
rs0 [primary] test> use lab4
switched to db lab4
rs0 [primary] lab4> let t0 = new Date(); t0
ISODate('2026-11-02T17:46:11.504Z')
rs0 [primary] lab4> db.part1.insertOne({ scenario: 1, note: "w: 3, wtimeout: 0" }, { writeConcern: { w: 3, wtimeout:
0 } })
```

Рисунок 3.1. Сценарій 1, частина 1: вузол 3 зупинено, вставка з `w: 3` без тайм-ауту почала чекати

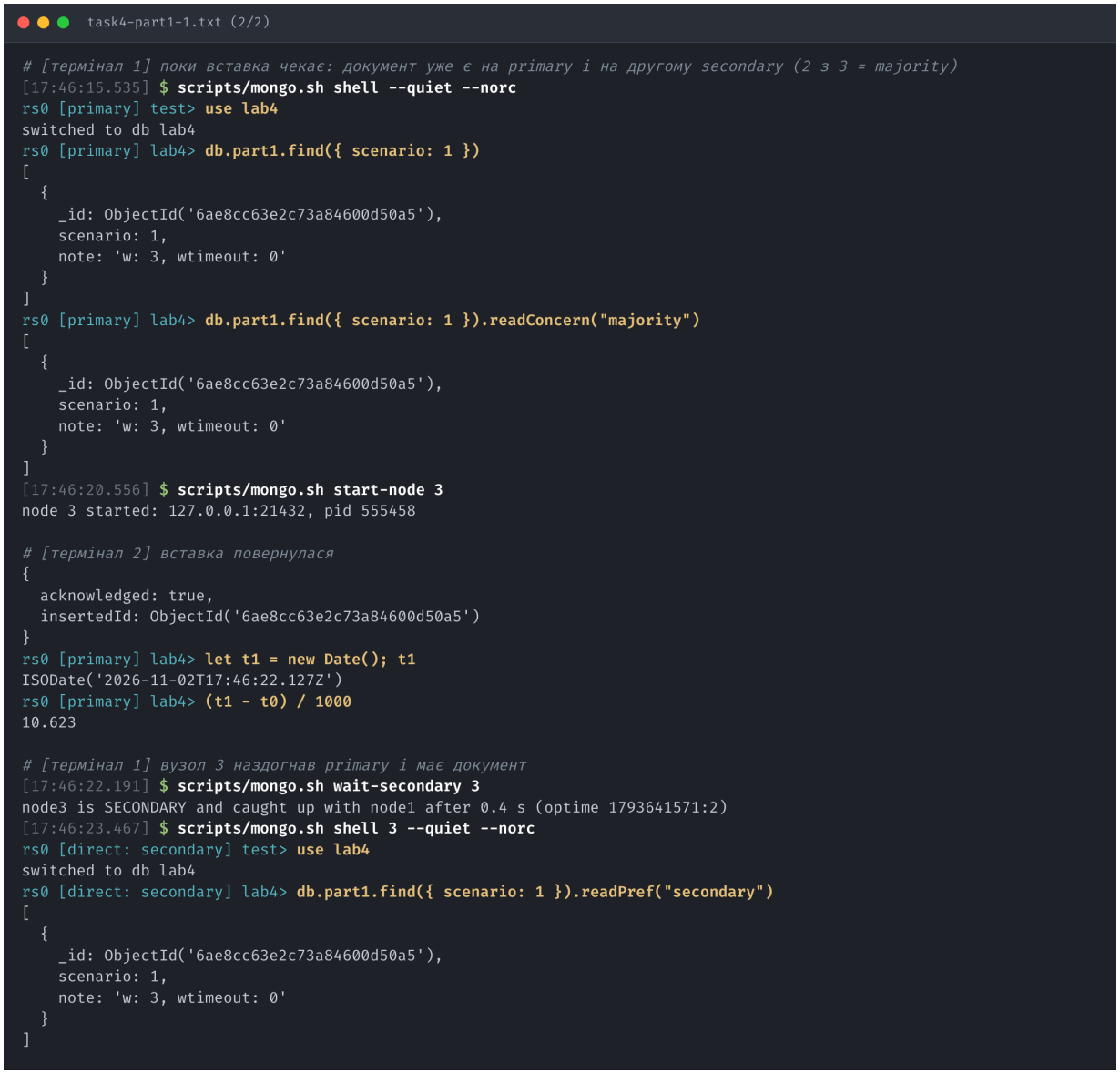


Рисунок 3.2. Сценарій 1, частина 2: читання під час очікування, повернення вузла 3, відповідь вставки через 10,623 с і документ на вузлі 3

3.3.2 Сценарій 2: `w: 3` з `wtimeout: 5000`

Вузол 3 знову зупинено, і та сама вставка йде з `wtimeout: 5000` (рис. 3.3). Через 5,049 с вона завершується помилкою `WriteConcernTimeout` (код 64), але у відповіді `n: 1`, і документ читається на primary, зокрема з `readConcern("majority")` (рис. 3.4). Він є і на node2. Після повернення вузол 3 наздогнав primary за 1,8 с і має той самий документ (рис. 3.5).

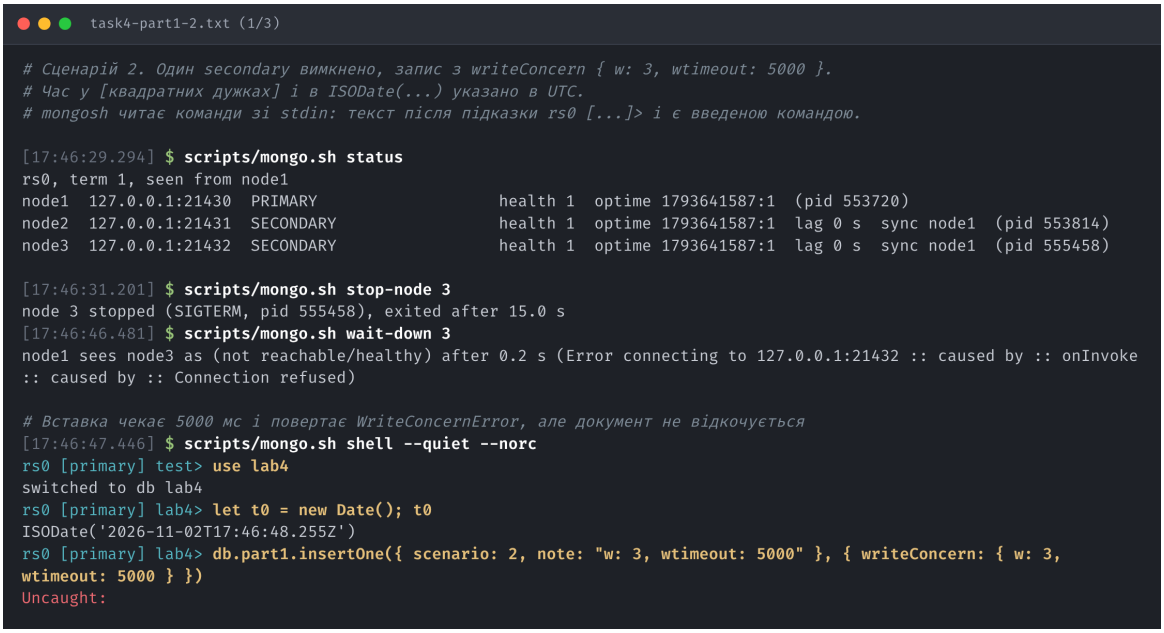


Рисунок 3.3. Сценарій 2, частина 1: зупинка вузла 3 і вставка з `w: 3, wtimeout: 5000`

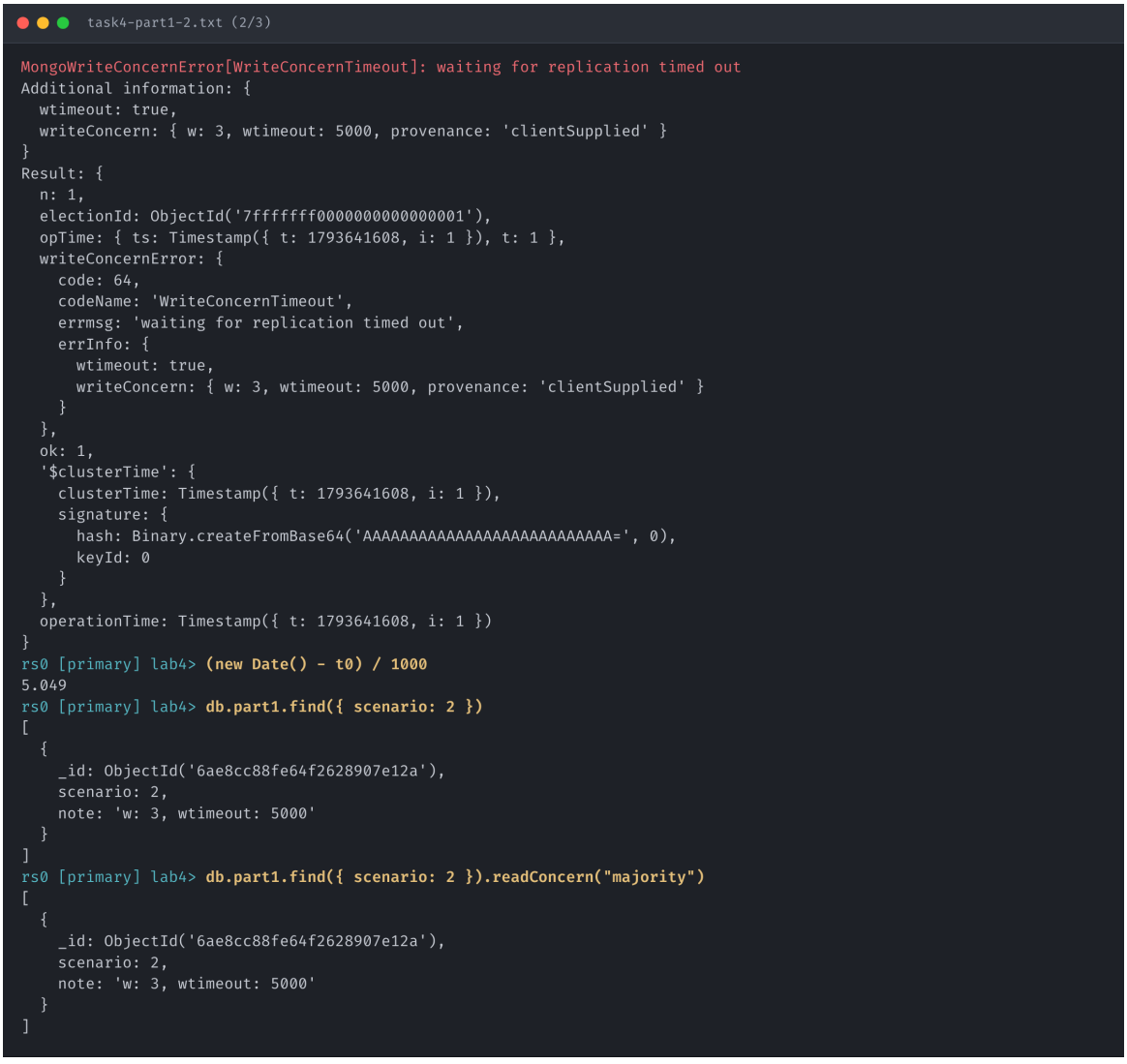


Рисунок 3.4. Сценарій 2, частина 2: через 5 с вставка повертає `WriteConcernTimeout` з `n: 1`, документ читається на primary з `majority`

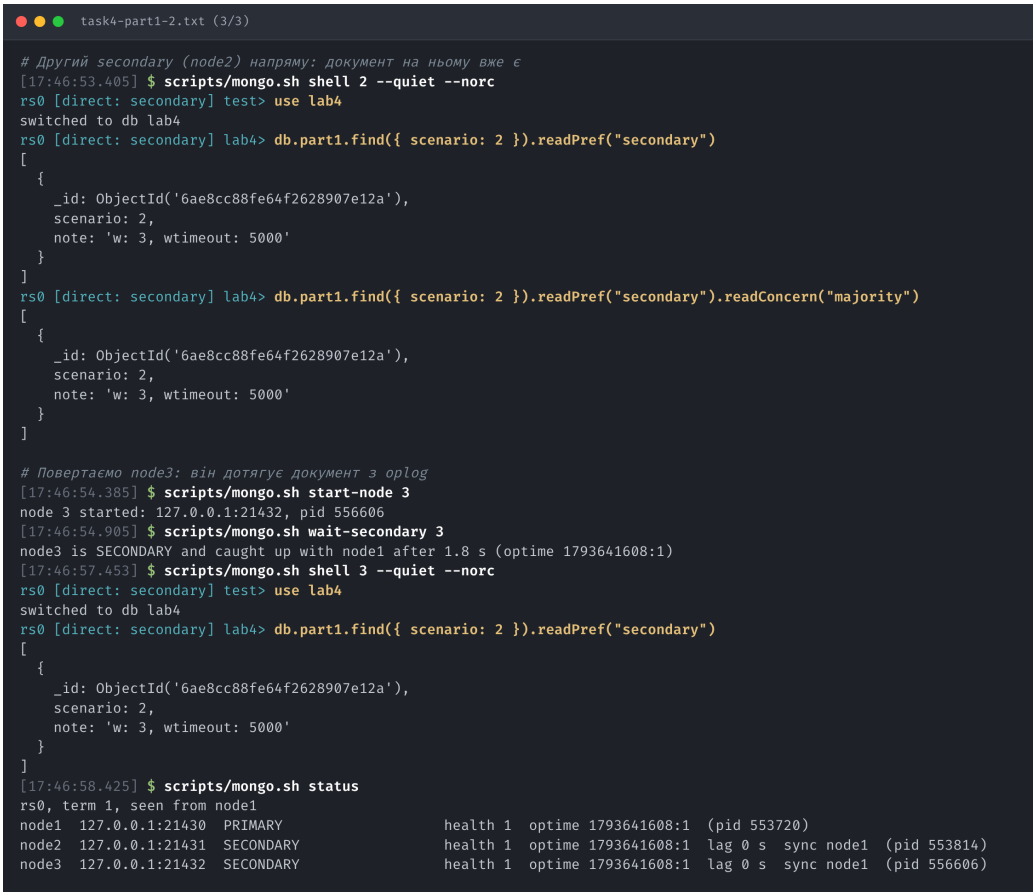


Рисунок 3.5. Сценарій 2, частина 3: документ на node2 з `majority` і на node3 після його повернення

3.3.3 Сценарій 3: перевибори primary

Поточний primary node1 штатно зупинено, новим primary став node2 у term 2 (рис. 3.6). Метрики виборів на node2 показують причину і час виборів, а через нього записано три документи з `w: "majority"` (рис. 3.7). Після повернення node1 приєднався як SECONDARY, наздогнав node2 за 1,7 с і має всі три документи (рис. 3.8).

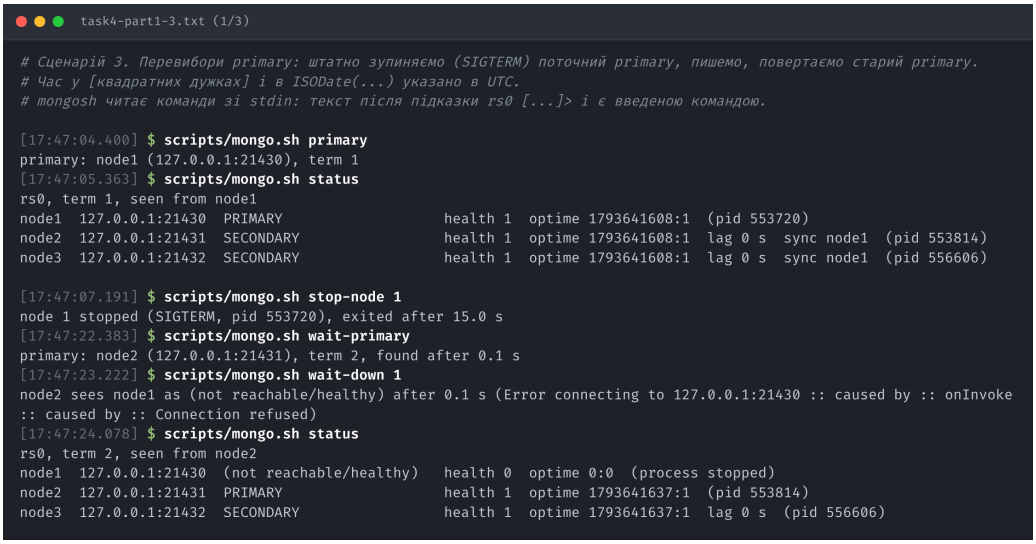


Рисунок 3.6. Сценарій 3, частина 1: зупинка primary node1, новий primary node2 у term 2

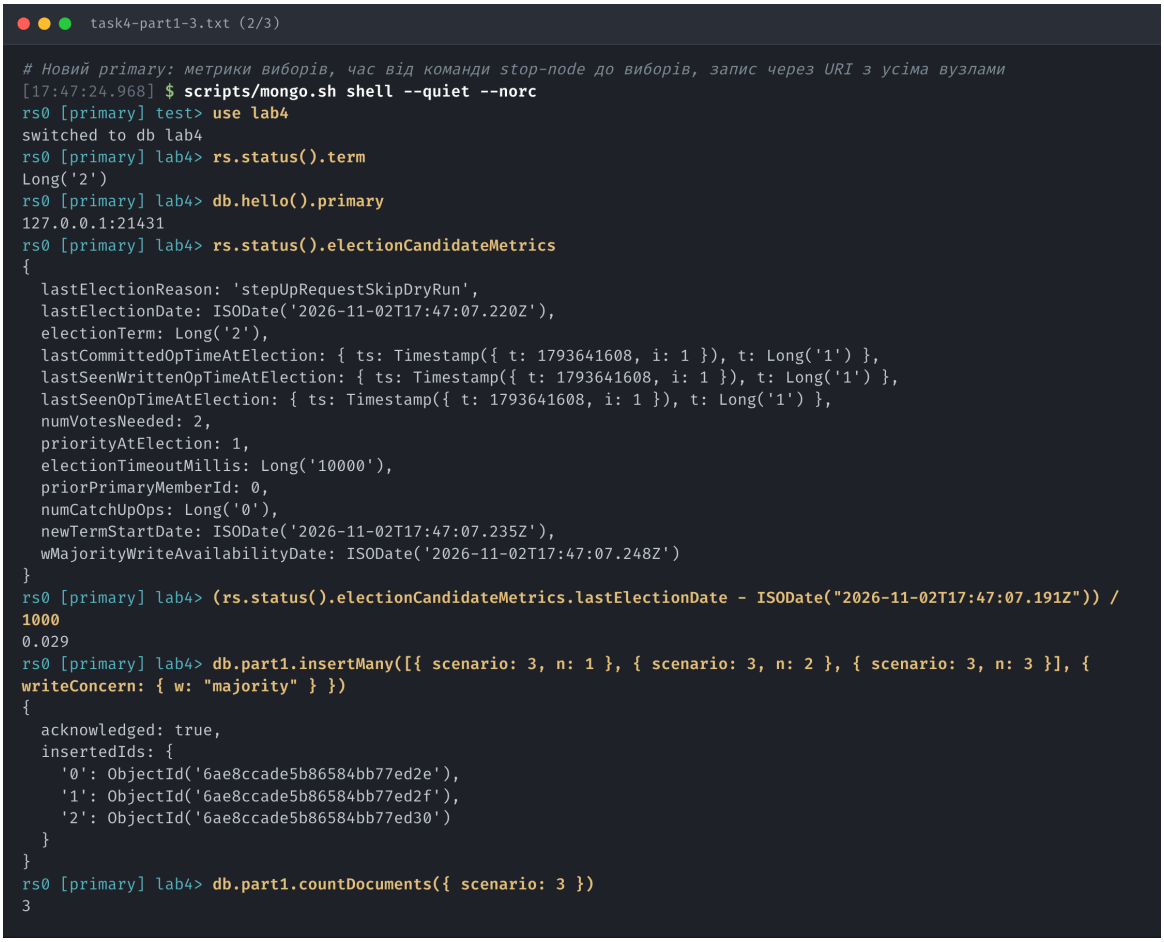


Рисунок 3.7. Сценарій 3, частина 2: метрики виборів на новому primary і запис трьох документів з `w: "majority"`

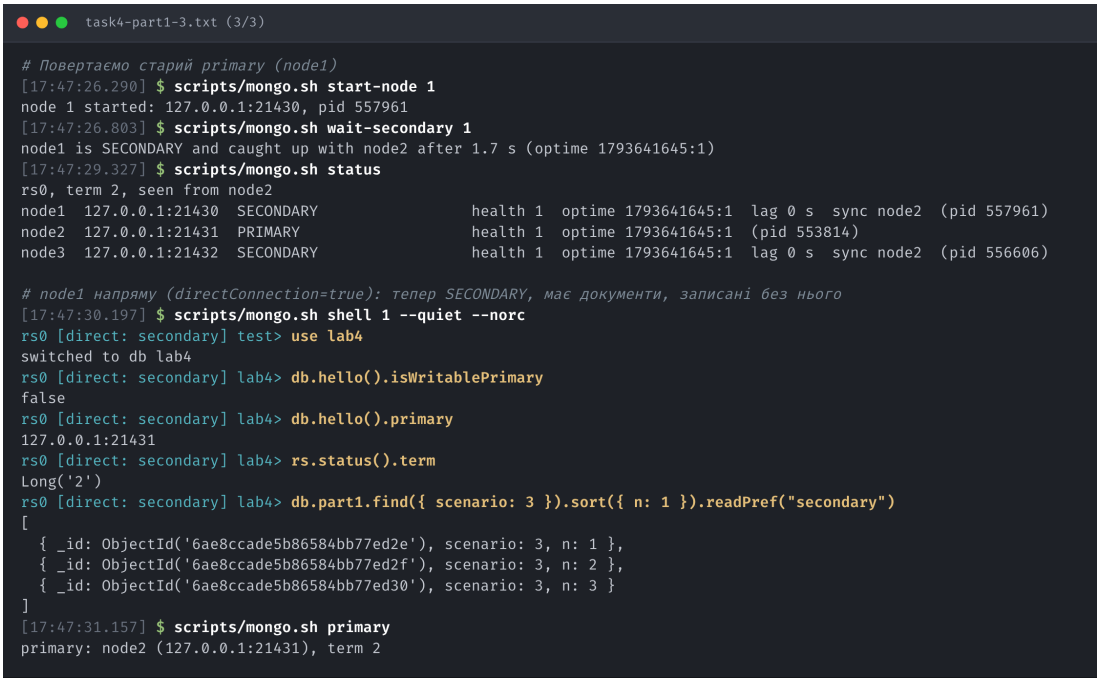


Рисунок 3.8. Сценарій 3, частина 3: старий primary повернувся як SECONDARY і має документи, записані без нього

3.4 Клієнт MongoDB і retryable writes

Для Swift використано драйвер MongoKitten [11]. Він знаходить primary за рядком підключення зі списком вузлів, але не реалізує retryable writes [8, 10]: його `findOneAndUpdate` не надсилає `txnNumber`, а сам драйвер нічого не повторює. Без них інкремент, відповідь на який загубилася разом з primary, не можна безпечно повторити: він міг бути вже застосований.

Тому клієнт `MongoCounterClient` (`Sources/CounterCore/`) сам будує `findAndModify`, як офіційні драйвери: власна логічна сесія з випадковим UUID (`lsid`) і наступний `txnNumber` на кожен логічний запис:

```
var command: Document = [
    "findAndModify": location.collection,
    "query": ["_id": documentID ?? location.documentID] as Document,
    "update": update,                  // ["$inc": ["likes": 1]]
    "new": true,
    "upsert": true,
    "writeConcern": (writeConcern ?? self.writeConcern).document,
]
if retryableWrites {
    command["lsid"] = ["id": sessionID] as Document
    command["txnNumber"] = lastTransactionNumber
        .add(1, ordering: .relaxed).newValue
}
```

Повтор надсилає ту саму команду з тим самим `txnNumber`. Сервер пам’ятає останній виконаний `txnNumber` кожної сесії (у `config.transactions`, що реплікується разом із самим записом) і на повтор уже виконаного запису повертає збережений результат. Повторюються лише тимчасові помилки за специфікацією [10], з паузою від 10 до 250 мс і лімітом 60 с:

```
while true {
    do {
        return MongoOutcome(value: try await operation(), retries: retries)
    } catch {
        guard MongoCommandFailure.isRetryable(error), clock.now < giveUp
        else { throw error }
        retries += 1
        if let failure = error as? MongoCommandFailure,
           failure.isNotPrimary { try? await cluster.reconnect() }
        try await Task.sleep(for: backoff)
        backoff = min(backoff * 2, Self.maximumBackoff)
    }
}
```

`reconnect()` потрібен, коли вузол відповів, що він більше не primary: MongoKitten оновлює відомості про вузли лише раз на 10 с. Читання `value()` іде з `readConcern: "majority"`, тож не бачить записів, які ще можуть бути відкочені.

3.5 Частина II: програма `mongo-bench`

`mongo-bench` (`Sources/mongo-bench/`) скидає лічильник у 0 з `w: "majority"`, створює 10 окремих клієнтів (у кожного свої з'єднання і своя сесія) і в `withThrowingTaskGroup` запускає по 10 000 інкрементів на клієнта. `ContinuousClock` вимірює лише фазу інкрементів.

З параметром `--kill-primary-after T` програма через T секунд знаходить primary командою `hello` і вбиває його через `scripts/mongo.sh kill-node` (SIGKILL, тобто раптова відмова без передачі ролі). Коли всі клієнти закінчили, вузол запускається знову, програма чекає, доки він стане SECONDARY і наздожене primary, робить один запис з `w: "majority"` в окремий документ, щоб усе попереднє стало majority-committed, і читає фінальне значення з `readConcern: "majority"`. Втраченими вважаються підтверджені сервером інкременти, яких немає у фінальному значенні.

Контрольний прогін `--no-retryable-writes` повторює інкременти після помилок так само, але без `lsid` і `txnNumber`.

3.6 Частина III: веб-лічильник на MongoDB

Сховище `MongoCounterStore` реалізує той самий протокол `CounterStore`, що й сховища робіт №1…3. `GET /inc` виконує описаний вище `findAndModify` з `$inc` і `w: "majority"`, `GET /count` читає значення з `readConcern: "majority"`. mongod обробляє запити одного з'єднання по черзі, а `txnNumber` однієї сесії має лише зростати, тому сховище тримає пул з 10 клієнтів і позичає кожному HTTP-запиту один з них:

```
public func increment() async throws {
    try await clients.withLease { _ = try await $0.increment() }
}
```

Сервер запускається так:

```
counter-server --store mongo --mongo-clients 10 --mongo-url "$MONGO_URL"
```

де `MONGO_URL` є наведеним вище рядком підключення з усіма трьома вузлами.

3.7 Методика вимірювань

1. `scripts/task4-part1.sh` виконав три сценарії частини I 02.11.2026 з 17:45 до 17:47 UTC. Логи вузлів збережено в `reports/logs/mongo-node{1,2,3}.log`.

2. `scripts/task4.sh` перезапускає replica set з логами в `reports/logs/task4/` і тричі проганяє `w1` і `majority`. Потім тричі прогони з kill: primary отримує SIGKILL на 35 % медіанного часу прогону без відмови (близько 8,1 с для `w1` і 35,7 с для `majority`). Після них один контрольний прогін без `lsid` і `txnNumber`. До і після кожного прогону скрипт читає `serverStatus().transactions.retriedStatementsCount` усіх вузлів: приріст показує, скільки повторів сервер упізнав як уже виконані. Для кожного прогону з kill він зберігає вивід `mongo-bench` і відповідні рядки журналів усіх трьох вузлів у `reports/logs/task4-failover-<прогін>-<повтор>.txt`. Результат записується в `reports/data/task4-bench.csv` (18:25…18:47 UTC).
3. Той самий скрипт запускає `counter-server --store mongo` і `loadgen` з 10 клієнтами по 10 000 запитів, тричі (18:47…18:56 UTC). Результат записується в `reports/data/task4-web.csv`. Числа для інших сховищ узято з `reports/data/task1.csv` (рядки з 10 клієнтами), `task2-web.csv` і `task3-web.csv`.
4. У таблицях наведено медіану трьох повторів, мінімум і максимум.

3.8 Аналітичний ноутбук

Дані оброблено в ноутбуці `solution.ipynb`, опублікованому разом зі звітом. Ноутбук показує протоколи частини I і витягує з них час і стани вузлів, читає `task4-bench.csv`, `task4-web.csv`, `task1.csv`, `task2-web.csv` і `task3-web.csv`, рахує в pandas медіани й діапазони для таблиць 4.1…4.4 і перевіряє, що вони збігаються зі звітом. Він розбирає журнали відмов (`reports/logs/task4-failover-*.txt`) і будує рисунки 4.1…4.3. Розділ «Жива перевірка» при `RUN_LIVE = True` піднімає replica set з трьох вузлів у тимчасовому каталозі, проганяє `mongo-bench` з `w: 1` і `w: "majority"` на 10 × 300 інкрементах і зупиняє вузли. Обидва прогони дали рівно 3 000. Числа звіту взято не з цього розділу, а з CSV.

4 Результати

4.1 Частина I: що показали сценарії

Write concern визначає лише момент, коли сервер відповідає клієнту, а не те, чи відбувся запис. У сценарії 1 вставка з `w: 3` чекала 10,623 с, з них близько 1,6 с після `start-node 3`: стільки вузол стартував і дотягнув документ з oplog. Якби вузол не повернувся, клієнт без `wtimeout` чекав би безкінечно. Весь цей час документ уже був на primary і на node2 і читався з `readConcern("majority")`: majority-committed означає «є на 2 з 3 вузлів», а не «підтверджено клієнту».

У сценарії 2 `WriteConcernTimeout` прийшов через 5,049 с, але документ залишився: є на primary, на node2 і після повернення на node3. За документацією MongoDB після `wtimeout` не скасовує вже виконаних змін [6], тому повтор такої вставки створив би другий документ. Документ був на 2 вузлах з 3, і тому читається з `readConcern: "majority"` [7].

У сценарії 3 штатна зупинка primary (SIGTERM) призвела до передачі ролі: `lastElectionReason: 'stepUpRequestSkipDryRun'`, вибори відбулися через 0,029 с після команди,

`w: "majority"` став доступний ще через 0,028 с. При раптовому SIGKILL (частина II) такої передачі немає, і secondary чекають `electionTimeoutMillis`, тобто 10 с. Старий primary повернувся як SECONDARY у term 2, наздогнав новий primary за 1,7 с і отримав три документи, записані без нього. Пріоритети вузлів однакові, тому роль primary лишилась на node2.

4.2 Частина II: час і точність лічильника

Таблиця 4.1. Час і пропускна здатність, 10 клієнтів по 10 000 інкрементів (медіана трьох повторів; контрольний прогін один)

Прогін	Медіана, с	Мін., с	Макс., с	Інкр./с
`w1`	22,49	22,00	29,18	4 447,0
`majority`	101,52	100,08	103,41	985,1
`w1-kill`	34,66	34,48	35,73	2 885,2
`majority-kill`	112,36	109,33	139,85	890,0
`majority-kill-noretry`	168,40	168,40	168,40	593,8

Таблиця 4.2. Фінальне значення, втрачені підтверджені інкременти, повтори клієнта і повтори, упізнані сервером, у кожному прогоні (очікується 100 000)

Прогін	Фінальне значення	Втрачено	Повтори	Упізнано сервером
`w1`	100 000 у всіх трьох	0; 0; 0	0; 0; 0	0; 0; 0
`majority`	100 000 у всіх трьох	0; 0; 0	0; 0; 0	0; 0; 0
`w1-kill`	99 996; 99 999; 100 000	4; 1; 0	480; 490; 450	0; 2; 0
`majority-kill`	100 000 у всіх трьох	0; 0; 0	470; 480; 426	10; 7; 9
`majority-kill-noretry`	100 010	10 зайвих	450	0

Без відмов обидва write concern дають рівно 100 000: `findOneAndUpdate` з `$inc` атомарно змінює документ на primary, тому втрачених оновлень, як у `IMap` без блокувань у роботі № 3, тут немає. `majority` у 4,5 раза повільніший за `w1` (101,52 проти 22,49 с). Середня затримка інкремента (10 клієнтів, поділені на пропускну здатність) 2,25 мс з `w: 1` і 10,15 мс з `w: "majority"`. З `w: 1` primary відповідає, щойно застосував запис у пам’яті. `w: "majority"` за замовчуванням включає `j: true`, тобто запис має потрапити в журнал на диску на більшості вузлів [6, 9], а `fsync` на цьому SSD займає 5,5…8,1 мс (робота № 1). Окремий `fsync` на кожен інкремент не потрібен: очікування write concern відбувається вже після того, як `$inc` застосовано, тож 10 клієнтів чекають спільних скидань журналу паралельно. 985,1 інкр./с у 8 разів більше, ніж дає файл з `fsync` на кожен інкремент (122,2 запитів/с).

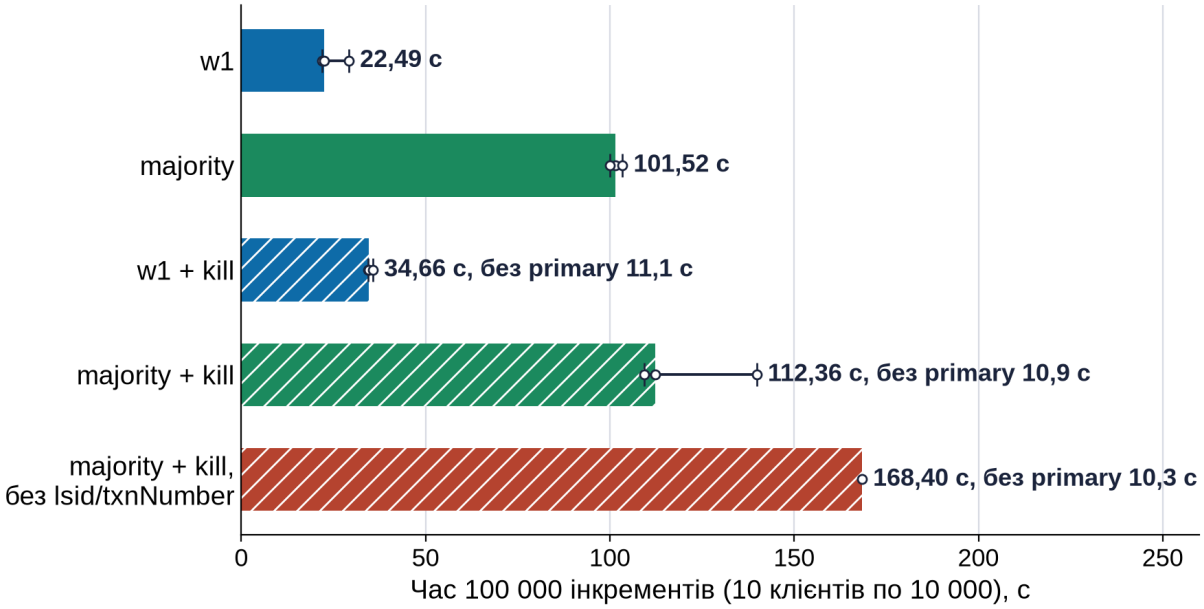


Рисунок 4.1. Час 100 000 інкрементів: медіана трьох повторів (стовпці), мінімум і максимум (відрізки), окремі повтори (точки); у прогонах з kill підписано медіану паузи без primary

Відмова primary коштує близько 11 с простою: від kill до першого підтвердженого інкременту 10,3…11,3 с у всіх семи прогонах. Secondary не отримують heartbeat від primary, чекають `electionTimeoutMillis` (10 000 мс) і проводять вибори. Прогони з kill довші за звичайні на 12,17 с (`w1`) і 10,84 с (`majority`) за медіаною. Усі 426…490 повторів припадають на цю паузу: 10 інкрементів, що були в дорозі, обриваються з `connectionClosed`, а далі кожен клієнт приблизно 4 рази за секунду отримує `noAvailableHosts`, доки primary немає. Помилок, на яких клієнт здався, немає в жодному прогоні. Великий розкид `majority-kill` (до 139,85 с) і повільний контрольний прогін (168,40 с) не пов’язані з виборами, пауза в них та сама (10,3…11,1 с). Імовірна причина в диску: розділ майже заповнений, а затримка `fsync` на ньому вже в роботі № 1 мала два рівні.

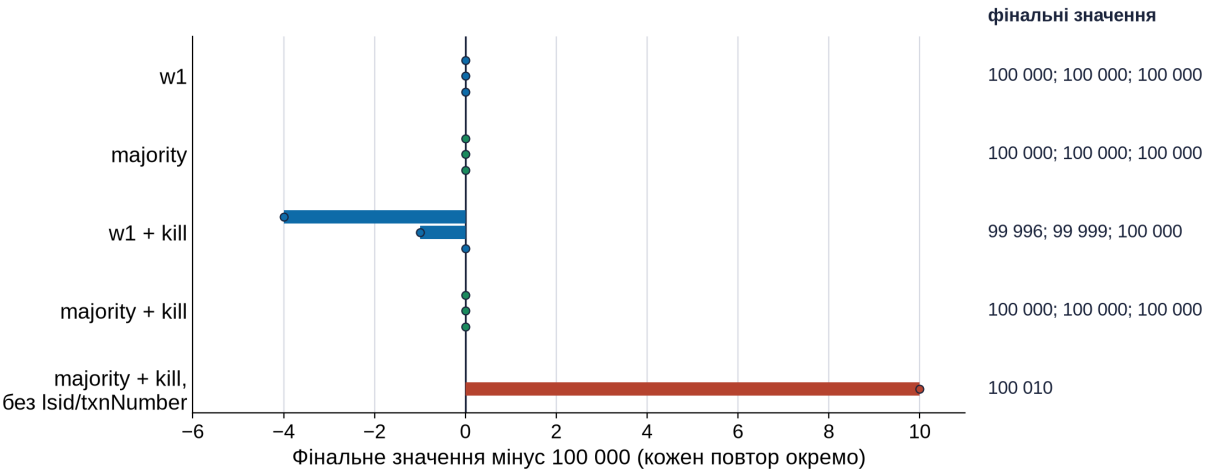


Рисунок 4.2. Відхилення фінального значення від 100 000 у кожному повторі: нуль означає точний лічильник, мінус втрачені підтверджені інкременти, плюс інкременти, пораховані двічі

4.3 Відмова primary: що стається з даними

Таблиця 4.3. Прогони з kill: вбитий вузол, новий primary і розрив між oplog нового primary (`targetOpTime` після виборів) і тим, що вбитий вузол мав на диску (`oplogTruncateAfterPoint` після перезапуску)

Прогін	Повтор	Kill, с	Вбито	Новий primary	Розрив, записів oplog	Втрачено
`w1-kill`	1	8,08	node2	node1, term 16	452	4
`w1-kill`	2	8,09	node2	node1, term 18	2 386	1
`w1-kill`	3	8,09	node2	node3, term 20	346	0
`majority-kill`	1	35,67	node1	node2, term 17	2	0
`majority-kill`	2	35,67	node1	node2, term 19	8	0
`majority-kill`	3	36,01	node3	node2, term 21	0	0

`w1-kill` втратив 4, 1 і 0 підтверджених інкрементів, разом 5 з 300 000. Як це стається, видно з таблиці 4.3 і журналів (уривок нижче). З `w: 1` primary відповідає клієнту до реплікації і до запису в журнал, а журнал без `j: true` скидається на диск раз на 100 мс [9]. Secondary натомість журналюють кожну застосовану пачку oplog. Тому після SIGKILL вбитий primary мав на диску на 346…2 386 записів oplog менше, ніж новий primary. Інкременти, які він встиг підтвердити, але не встиг віддати жодному secondary, не лишилися ніде: їх немає ні на новому primary, ні в журналі старого. Через це немає і rollback [12]: у журналах жодного рядка ROLLBACK, файлів rollback немає. Старий primary після перезапуску наздогнав новий як SECONDARY. Якби він встиг записати ці інкременти в журнал, вони стали б розбіжною гілкою oplog і були б відкочені в rollback-файли, але в лічильнику їх однаково не було б.

Уривок журналів трьох вузлів для `w1-kill`, повтор 1 (UTC, атрибути скорочено):

```
18:35:05.542 node1 Member is now in state DOWN  127.0.0.1:21431
18:35:16.556 node1 Starting an election, since we've seen no PRIMARY
             in election timeout period  {electionTimeoutPeriodMillis:10000}
18:35:16.561 node1 Election succeeded, assuming primary role  {term:16}
18:35:16.562 node1 Caught up to the latest optime known via heartbeats
             {targetOpTime: 1793644505:11766}
18:35:32.102 node2 ***** SERVER RESTARTED *****
18:35:32.856 node2 Incrementing the rollback ID after unclean shutdown
18:35:32.863 node2 Removing unapplied oplog entries after
             oplogTruncateAfterPoint  {1793644505:11314}
18:35:34.781 node2 Replica set state transition SECONDARY
```

`majority-kill` дав рівно 100 000 у всіх трьох повторах. Запис з `majority` підтверджується лише тоді, коли він є в журналі більшості вузлів, а нового primary можна

обрати лише з вузлів, що мають усі majority-committed записи. Тому жоден підтверджений інкремент не може зникнути. Розрив oplog тут 0…8 записів, бо кожен такий запис примусово синхронізує журнал [6].

Ще є 10 інкрементів, які були в дорозі в момент kill: клієнт не отримав відповіді і не знає, чи їх застосовано. Сервер упізнав 10, 7 і 9 з них як уже виконані: вони встигли потрапити на secondary, що став primary, разом з записом сесії в `config.transactions`, і новий primary повернув збережений результат. Решту (0, 3 і 1) новий primary виконав уперше. У контрольному прогоні без `lsid` і `txnNumber` сервер не зміг відрізнити ті самі 10 повторів від нових записів, і лічильник показав 100 010. Для точного лічильника при відмові primary потрібні і `w: "majority"`, і retryable writes.

4.4 Частина III: порівняння сховищ веб-лічильника

Таблиця 4.4. Веб-лічильник, 10 клієнтів по 10 000 запитів `GET /inc` (медіана трьох повторів, у всіх прогонах фінальне значення 100 000)

Сховище	Загальний час, с	Мін., с	Макс., с	Запитів/с
In-memory	3,20	2,92	3,31	31 256,0
Disk (`fsync`)	818,08	814,26	824,12	122,2
PostgreSQL	302,44	199,76	306,17	330,6
Hazelcast (CP)	8,88	8,79	9,08	11 259,7
MongoDB (`majority`)	133,59	121,80	156,01	748,6

Дані для пам’яті й диска взято з `reports/data/task1.csv`, для PostgreSQL з `task2-web.csv`, для Hazelcast з `task3-web.csv`, для MongoDB з `task4-web.csv`.

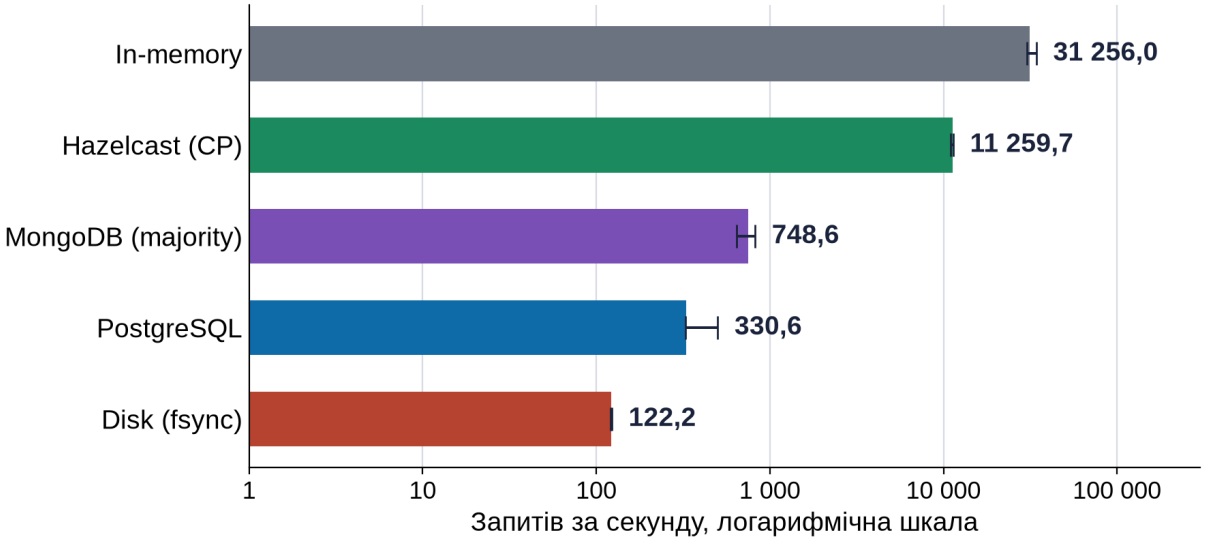


Рисунок 4.3. Пропускна здатність веб-лічильника для п’яти сховищ, логарифмічна шкала: медіана трьох повторів (стовпці), мінімум і максимум (відрізки)

MongoDB з `w: "majority"` дав 748,6 запитів/с. Це в 2,26 раза більше за PostgreSQL і в 6,1 раза більше за файл з `fsync`. Усі три сховища чекають на диск при кожному інкременті, різниця в тому, хто чекає паралельно. У файловому сховищі `Mutex` тримається разом з `fsync`. У PostgreSQL блокування рядка тримається до `COMMIT`, а `COMMIT` чекає `fsync` WAL, тож інкременти одного рядка йдуть строго по черзі (робота № 2). У MongoDB `$inc` застосовується і звільняє документ одразу, а очікування журналу на більшості вузлів іде вже після цього, тому 10 запитів чекають одночасно.

MongoDB у 15 разів повільніший за Hazelcast і в 41,8 раза за пам'ять. Hazelcast без persistence підтверджує запис після реплікації в пам'ять інших вузлів, а MongoDB з `majority` після запису в журнал на диску двох вузлів з трьох. Тому дані MongoDB зберігаються навіть після вимкнення всієї машини, а CP Subsystem Hazelcast без persistence витримує лише втрату вузла.

Веб-варіант повільніший за прямий `mongo-bench majority` (748,6 проти 985,1 інкр./с) через HTTP між `loadgen` і `counter-server`. Розкид між повторами від 121,80 до 156,01 с (25,6 % від медіани, у Hazelcast 3 %). Причина, як і в `majority-kill`, у затримці `fsync` на майже заповненому диску.

5 Висновки

1. Replica set MongoDB 8.2.12 з трьох вузлів (P-S-S) розгорнуто окремими процесами на одній машині. Write concern визначає лише момент відповіді: `w: 3` без тайм-ауту чекав повернення вузла 10,623 с, а з `wtimeout: 5000` повернув `WriteConcernTimeout` через 5,049 с, проте документ лишився записаним на двох вузлах і читався з `readConcern: "majority"`.
2. Штатна зупинка primary передала роль за 0,029 с (`stepUpRequestSkipDryRun`), раптова (SIGKILL) зупинила запис на 10,3…11,3 с, бо secondary чекають `electionTimeoutMillis` 10 с. В усіх випадках старий primary повернувся як SECONDARY з усіма даними, записаними без нього.
3. Без відмов `findOneAndUpdate` з `$inc` точний при обох write concern: 100 000 за 22,49 с з `w: 1` (4 447,0 інкр./с) і за 101,52 с з `w: "majority"` (985,1 інкр./с). `majority` у 4,5 раза повільніший, бо кожен інкремент чекає журналу на диску двох вузлів.
4. З `w: 1` раптова відмова primary втратила 4, 1 і 0 підтверджених інкрементів: primary підтверджував їх до реплікації і до запису в журнал, тому вони зникли без rollback. З `w: "majority"` і retryable writes результат рівно 100 000 у всіх трьох прогонах з відмовою; сервер упізнав 7…10 повторів як уже виконані. Без `lsid` і `txnNumber` той самий сценарій дав 100 010.
5. Як сховище веб-лічильника MongoDB з `w: "majority"` дав 748,6 запитів/с: у 2,26 раза швидше за PostgreSQL (330,6) і в 6,1 раза за файл з `fsync` (122,2), бо інкременти чекають диска паралельно, але в 15 разів повільніше за Hazelcast (11 259,7) і в 41,8 раза за пам'ять (31 256,0).

6 Джерела

1. MongoDB Manual. Replication [Електронний ресурс]. Режим доступу: https://www.mongodb.com/docs/manual/replication/ (дата звернення: 02.11.2026).
2. MongoDB Manual. Three Member Replica Sets [Електронний ресурс]. Режим доступу: https://www.mongodb.com/docs/manual/core/replica-set-architecture-three-members/ (дата звернення: 02.11.2026).
3. MongoDB Manual. Deploy a Replica Set for Testing and Development [Електронний ресурс]. Режим доступу: https://www.mongodb.com/docs/manual/tutorial/deploy-replica-set-for-testing/ (дата звернення: 02.11.2026).
4. MongoDB Manual. Replica Set Elections [Електронний ресурс]. Режим доступу: https://www.mongodb.com/docs/manual/core/replica-set-elections/ (дата звернення: 02.11.2026).
5. MongoDB Manual. db.collection.findOneAndUpdate() [Електронний ресурс]. Режим доступу: https://www.mongodb.com/docs/manual/reference/method/db.collection.findOneAndUpdate/ (дата звернення: 02.11.2026).
6. MongoDB Manual. Write Concern [Електронний ресурс]. Режим доступу: https://www.mongodb.com/docs/manual/reference/write-concern/ (дата звернення: 02.11.2026).
7. MongoDB Manual. Read Concern “majority” [Електронний ресурс]. Режим доступу: https://www.mongodb.com/docs/manual/reference/read-concern-majority/ (дата звернення: 02.11.2026).
8. MongoDB Manual. Retryable Writes [Електронний ресурс]. Режим доступу: https://www.mongodb.com/docs/manual/core/retryable-writes/ (дата звернення: 02.11.2026).
9. MongoDB Manual. Journaling [Електронний ресурс]. Режим доступу: https://www.mongodb.com/docs/manual/core/journaling/ (дата звернення: 02.11.2026).
10. MongoDB Specifications. Retryable Writes [Електронний ресурс]. Режим доступу: https://github.com/mongodb/specifications/blob/master/source/retryable-writes/retryable-writes.md (дата звернення: 02.11.2026).
11. MongoKitten [Електронний ресурс]. Режим доступу: https://github.com/orlandos-nl/MongoKitten (дата звернення: 02.11.2026).
12. MongoDB Manual. Rollbacks During Replica Set Failover [Електронний ресурс]. Режим доступу: https://www.mongodb.com/docs/manual/core/replica-set-rollbacks/ (дата звернення: 02.11.2026).